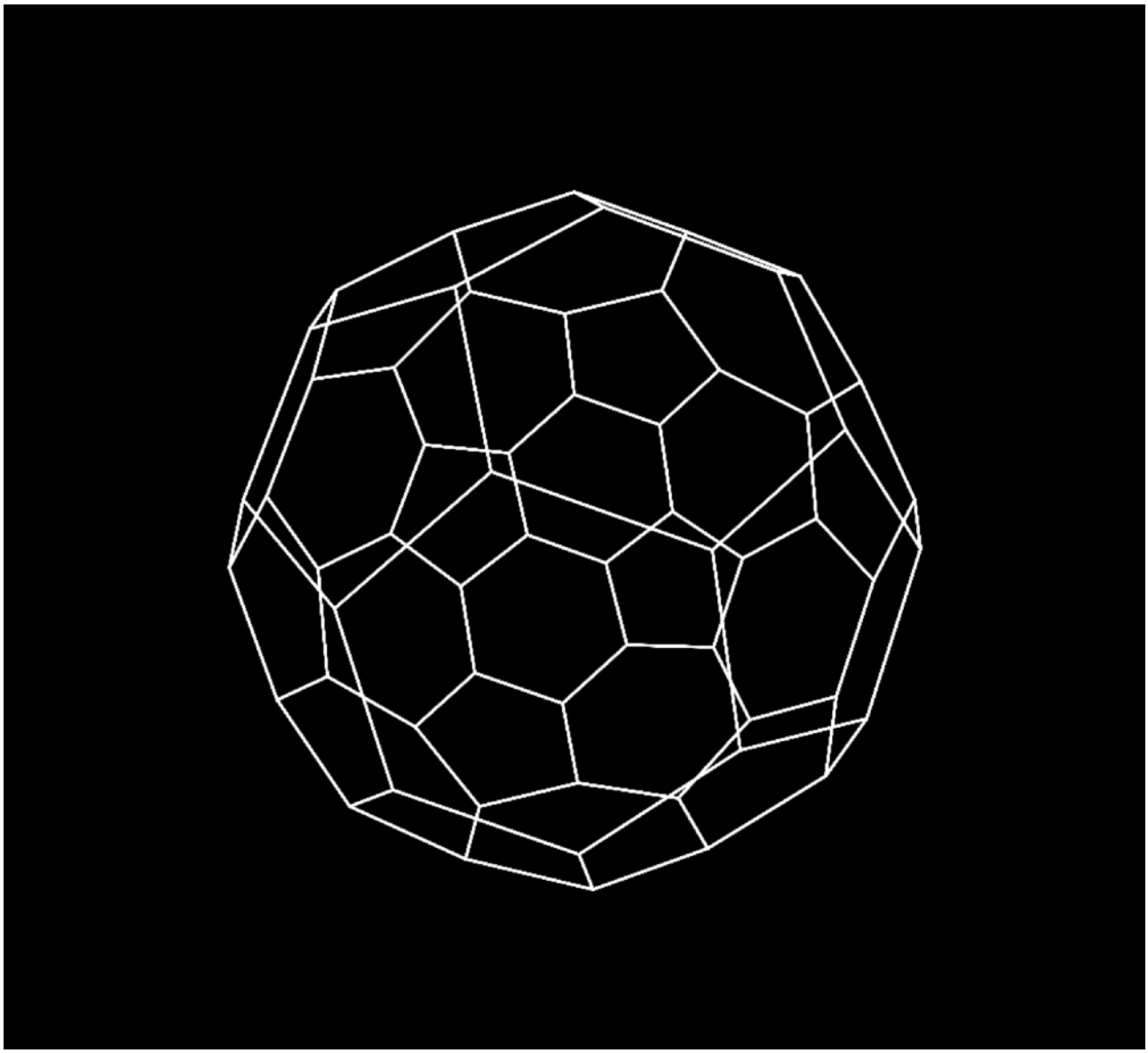
C Project

CO1020

G70 | E22131 | E22323



Task 1

3.1.1

```c
typedef struct {
    int width, height;
    float **pixels;
} canvas_t;
```

Making canvas_t structure: To use a 2D array we have used a pointer to pointer variable because we cannot define the size of the array when canvas_t is creating.

```c
canvas_t *create_canvas(int w, int h) {
    canvas_t *c = malloc(sizeof *c);
    c->width = w; c->height = h;
    c->pixels = malloc(h * sizeof *c->pixels);
    for (int y = 0; y < h; y++)
        c->pixels[y] = calloc(w, sizeof(float));
    return c;
}
```

When making the function to create canvas, it returns the address of the created canvas_t struct variable. Used malloc to allocate memories for each height of the canvas and used calloc to allocate memories for a number of width of the canvas and assign 0 for all pixels initially.

```c
void set_pixel_f(canvas_t *c, float x, float y, float intensity) {
    int x0 = floorf(x), y0 = floorf(y);
    float dx = x - x0, dy = y - y0;
    float w[2][2] = {
        {(1-dx)*(1-dy), dx*(1-dy)},
        {(1-dx)*dy,      dx*dy}
    };
    for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
        int xi = x0 + j, yi = y0 + i;
        if (xi >= 0 && xi < c->width && yi >= 0 && yi < c->height) {
            float old = c->pixels[yi][xi];
            c->pixels[yi][xi] = clampf(old + intensity * w[i][j], 0.0f, 1.0f);
        }
    }
}
static float clampf(float v, float a, float b) {
    return v < a ? a : v > b ? b : v;
}
```

Making set_pixel_f: Void function to set intensity values of given address of the canvas.
BILINEAR FILTERING: Used bilinear filtering to spread the intensity to four nearest pixels with respectively to the distance to the given floating point coordinate. When the distance to the nearest pixel with the given floating point coordinate is low, the intensity to the nearest pixel is high and vice versa. To do that we need to count the coordinates of nearest four pixels and the distances.
Used an additional clamping function to keep the intensity value of a pixel in between 0.0 and 1.0. Used an if statement to change the intensity of a pixel only if the pixel exists in the canvas.

```
void draw_line_f(canvas_t *c, float x0, float y0, float x1, float y1, float thickness) {
    float dx = x1 - x0, dy = y1 - y0;
    float len = fmaxf(fabsf(dx), fabsf(dy)) * 2.0f;
    int steps = (int)len + 1;
    float sx = dx / steps, sy = dy / steps;
    for (int i = 0; i <= steps; i++) {
        float cx = x0 + sx * i, cy = y0 + sy * i;
        for (float oy = -thickness/2; oy <= thickness/2; oy += 0.5f)
            for (float ox = -thickness/2; ox <= thickness/2; ox += 0.5f)
                set_pixel_f(c, cx + ox, cy + oy, 1.0f);
    }
}
```

draw_line_f: DDA: Used digital differential analyzing algorithm to count the length of the line and the number of pixels the line have. To do that we need to count the x difference, y difference, the greatest from them. The greatest from the x difference or y difference is set as the length of the line. Sx and sy counted as the difference between two steps of x and y directions.
To make the line draw with a thickness, in each step of drawing the line the pixels were set as a circle, radius of an half of the thickness.

```
void destroy_canvas(canvas_t *c) {
    for (int y = 0; y < c->height; y++)
        free(c->pixels[y]);
    free(c->pixels);
    free(c);
}
```

We have made a function named destroy_canvas to free up the allocated memory for the pixels

3.1.2 DEMO

```
void save_canvas_as_pgm(canvas_t *canvas, const char *filename) {
    FILE *fp = fopen(filename, "w");
    fprintf(fp, "P2\n%d %d\n255\n", canvas->width, canvas->height);
    for (int y = 0; y < canvas->height; y++) {
        for (int x = 0; x < canvas->width; x++) {
            int val = (int)(canvas->pixels[y][x] * 255.0f);
            fprintf(fp, "%d ", val);
        }
        fprintf(fp, "\n");
    }
    fclose(fp);
}
```

We have made a function to make a .pgm file from our canvas. We have used 255 to represent the color white. Then the 0 will represent the black. All other colors between black and white will represented by the relevant value between 0-255. Used basic File I/O too write on the .pgm file.

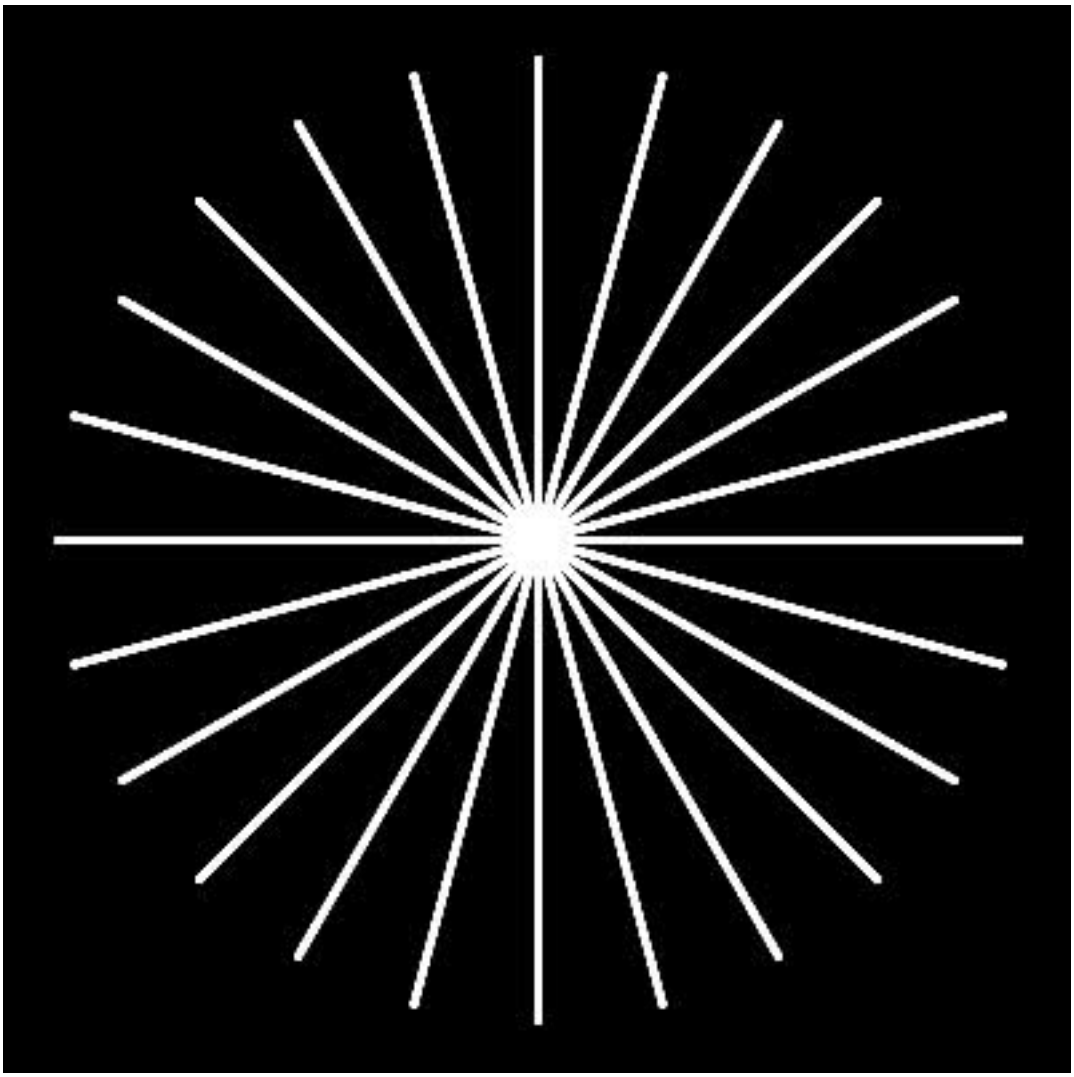
```
    float center_x = WIDTH / 2.0f;
    float center_y = HEIGHT / 2.0f;
    float radius = WIDTH * 0.45f;

    for (int angle_deg = 0; angle_deg < 360; angle_deg += 15) {
        float angle_rad = angle_deg * (PI / 180.0f);
        float end_x = center_x + cosf(angle_rad) * radius;
        float end_y = center_y + sinf(angle_rad) * radius;

        draw_line_f(canvas, center_x, center_y, end_x, end_y, 1.5f);
    }

    save_canvas_as_pgm(canvas, "clockface.pgm");
    destroy_canvas(canvas);
```

To draw the lines like the clock face, we have used the simple mathematical theories: we can obtain the functions to get the coordinates, by thinking the center coordinate as the origin of a cartesian plane. Multiplication of the radius and the cos(angle) gives the difference between the origin(center) x coordinate and the ending x coordinate of the line-end. Y axis is also can be think like that.
In for loop we have drawn the each line by using the draw_line_f function by giving the start of the line as the center of the canvas. The end of the line is set close to the edges of the canvas. Used basic trigonometry to find the line ending coordinates with the difference of 15 degrees. Then used Destroy_canvas function to free up all the allocated memory. Defined the width and height as 400x400



TASK 2

```
typedef struct {
    float x, y, z;
} vec3_t;
```

We have made all the function to work with cartesian coordinations. Thus we use this structure to store the cartesian data. Thus we only need to implement a function to convert spherical to cartesian only.

```
vec3_t vec3_from_spherical(float r, float theta, float phi) {
    vec3_t v;
    v.x = r * sinf(phi) * cosf(theta);
    v.y = r * sinf(phi) * sinf(theta);
    v.z = r * cosf(phi);
    return v;
}
```

We have used the simple trigonometry maths to convert spherical coordinates into cartesian coordinates: since phi is measured from z axis, r.sin(phi) gives the distance from origin to the reflection point on the x-y plane of the original coordinate, then we can divide them as x or y by multiplying with cos or sin phi respectively. Also, since phi is measured from the z axis. r.cos(phi) gives the reflection of the z coordinate on the z axis.

```
vec3_t vec3_normalize_fast(vec3_t v) {
    float length_sq = v.x*v.x + v.y*v.y + v.z*v.z;
    if (length_sq == 0.0f) return v;

    // Fast inverse square root approximation
    union { float f; int i; } conv = { length_sq };
    conv.i = 0x5f3759df - (conv.i >> 1);
    float inv_sqrt = conv.f * (1.5f - 0.5f * length_sq * conv.f * conv.f);

    vec3_t norm = { v.x * inv_sqrt, v.y * inv_sqrt, v.z * inv_sqrt };
    return norm;
```

vec3_normalize_fast is used to normalize a 3D vector v, but in a faster way than usual. First it calculates the square of the vector's length by adding up the squares of x, y, and z. If that length is zero, it just returns the vector as it is, since dividing by zero would be a problem. Then comes the clever part — instead of doing a normal square root (which is slow), it uses a trick with bit manipulation, known as fast inverse square root, to approximate 1/sqrt(length_sq). It uses a "magic number" and some math to make a pretty close guess. Then it multiplies that guess with each component of the vector to make it unit length. It's not 100% accurate but it's fast and usually good enough.

```c
vec3_t vec3_slerp(vec3_t a, vec3_t b, float t) {
    // Normalize inputs
    vec3_t v0 = vec3_normalize_fast(a);
    vec3_t v1 = vec3_normalize_fast(b);

    float dot = vec3_dot(v0, v1);
    if (dot > 0.9995f) {
        // Linear interpolation fallback for very close vectors
        vec3_t result = {
            v0.x + t*(v1.x - v0.x),
            v0.y + t*(v1.y - v0.y),
            v0.z + t*(v1.z - v0.z)
        };
        return vec3_normalize_fast(result);
    }

    if (dot < -1.0f) dot = -1.0f;
    else if (dot > 1.0f) dot = 1.0f;

    float theta = acosf(dot) * t;
    vec3_t v2 = {
        v1.x - v0.x * dot,
        v1.y - v0.y * dot,
        v1.z - v0.z * dot
    };
    v2 = vec3_normalize_fast(v2);

    vec3_t result = {
        v0.x * cosf(theta) + v2.x * sinf(theta),
        v0.y * cosf(theta) + v2.y * sinf(theta),
        v0.z * cosf(theta) + v2.z * sinf(theta)
    };
    return result;
}
```

This function does spherical linear interpolation (slerp) between two vectors. First, it normalize both vectors using that fast inverse square root trick, so they sit on the unit sphere. Then it finds the angle between them by taking the dot product. If the vectors are pretty much the same (dot product really close to 1), it just does a simple linear interpolation and normalize it after, that way is faster and more stable when angle is small. For the rest of the cases, it makes sure the dot product is clamped between -1 and 1, otherwise the acosf might go wrong. Then it figure out the actual angle, multiplies it by t, and builds a new vector pointing from the first vector towards the second, but without the part that's in the same direction as the first one. That new direction vector gets normalized too, and finally, it blends the two vectors using sine and cosine, so the result moves smoothly along the arc on the unit sphere. The output vector keeps a steady angle speed and always stays on the sphere.

```c
typedef struct {
    float m[4][4];
} mat4_t;
```

The structure mat4_t is made as a 2d array 4x4.

```c
// Identity matrix
mat4_t mat4_identity(void) {
    mat4_t m = {0};
    for (int i = 0; i < 4; i++) {
        m.m[i][i] = 1.0f;
    }
    return m;
}
```

Makes an identity matrix by filling the main diagonal of the 4x4 matrix by ones. It is a void function because it's output is always same

```c
// Scale matrix
mat4_t mat4_scale(float sx, float sy, float sz) {
    mat4_t m = mat4_identity();
    m.m[0][0] = sx;
    m.m[1][1] = sy;
    m.m[2][2] = sz;
    return m;
}
```

This function makes a 4x4 scale matrix. It first starts with an identity matrix, then it sets the diagonal values to sx, sy, and sz. That means when you multiply a vector with this matrix, it will get scaled along x, y, and z axis by those values.

```c
// Translation matrix
mat4_t mat4_translate(float tx, float ty, float tz) {
    mat4_t m = mat4_identity();
    m.m[0][3] = tx;
    m.m[1][3] = ty;
    m.m[2][3] = tz;
    return m;
}
```

This function makes a 4x4 translation matrix. It starts with the identity matrix, then puts tx, ty, and tz into the last column. That tells how much to move something in x, y, and z direction. So when you multiply it with a vector, it just moves the vector to a new position.

```c
// Matrix multiplication: result = a * b
mat4_t mat4_mul(mat4_t a, mat4_t b) {
    mat4_t result = {0};
    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            for (int k = 0; k < 4; k++) {
                result.m[row][col] += a.m[row][k] * b.m[k][col];
            }
        }
    }
    return result;
}
```

This function multiplies two 4x4 matrices a and b. It goes through each row and column and does the usual matrix multiply thing like adding up the products of row from a and column from b. It stores everything into result. At the end, you get a new matrix that kinda combines the effects of both a and b. It's useful when you wanna chain transforms like rotate then move something.

```c
// Multiply matrix by vector (assumes w=1)
vec3_t mat4_mul_vec3(mat4_t m, vec3_t v) {
    vec3_t out;
    float x = v.x, y = v.y, z = v.z;

    out.x = m.m[0][0]*x + m.m[0][1]*y + m.m[0][2]*z + m.m[0][3];
    out.y = m.m[1][0]*x + m.m[1][1]*y + m.m[1][2]*z + m.m[1][3];
    out.z = m.m[2][0]*x + m.m[2][1]*y + m.m[2][2]*z + m.m[2][3];

    return out;
}
```

This function takes a 3D vector v and multiplies it with a 4x4 matrix m. It basically applies the transformation from the matrix to the vector — like rotating, scaling, or moving it. It grabs the x, y, z from the vector, does some multiply and adding with the matrix values, and gives back a new vector out. It's like sending the vector through a machine that change its position or direction.

```c
// Rotation matrix (Euler angles, radians) around X, Y, then Z axes
mat4_t mat4_rotate_xyz(float rx, float ry, float rz) {
    mat4_t rxm = mat4_identity();
    mat4_t rym = mat4_identity();
    mat4_t rzm = mat4_identity();

    float cx = cosf(rx);
    float sx = sinf(rx);
    float cy = cosf(ry);
    float sy = sinf(ry);
    float cz = cosf(rz);
    float sz = sinf(rz);

    // Rotation X
    rxm.m[1][1] = cx; rxm.m[1][2] = -sx;
    rxm.m[2][1] = sx; rxm.m[2][2] = cx;

    // Rotation Y
    rym.m[0][0] = cy; rym.m[0][2] = sy;
    rym.m[2][0] = -sy; rym.m[2][2] = cy;

    // Rotation Z
    rzm.m[0][0] = cz; rzm.m[0][1] = -sz;
    rzm.m[1][0] = sz; rzm.m[1][1] = cz;

    // Combine rotations Z * Y * X
    return mat4_mul(rzm, mat4_mul(rym, rxm));
}
```

This function makes a rotation matrix from 3 angles: rx, ry, and rz which is rotations around X, Y, and Z axis. It first makes 3 identity matrices for each axis, then fills in the values using sine and cosine to do the actual rotation math. Each matrix rotates around one axis. Then at the end, it multiplies them together in the order Z * Y * X, so all the rotations gets combined into one. It's like building a big spin move from 3 smaller ones.

3.2.2 DEMO

```c
#include <stdio.h>
#include <math.h>
#include "math3d.h"

vec3_t multiply_point(mat4_t M, vec3_t v) {
    vec3_t r;
    float x = v.x, y = v.y, z = v.z;
    float w = M.m[3][0]*x + M.m[3][1]*y + M.m[3][2]*z + M.m[3][3];
    r.x = (M.m[0][0]*x + M.m[0][1]*y + M.m[0][2]*z + M.m[0][3]) / w;
    r.y = (M.m[1][0]*x + M.m[1][1]*y + M.m[1][2]*z + M.m[1][3]) / w;
    r.z = (M.m[2][0]*x + M.m[2][1]*y + M.m[2][2]*z + M.m[2][3]) / w;
    return r;
}

int main(void) {
    vec3_t cube[8] = {
        {-0.5f,-0.5f,-0.5f},{ 0.5f,-0.5f,-0.5f},
        { 0.5f, 0.5f,-0.5f},{-0.5f, 0.5f,-0.5f},
        {-0.5f,-0.5f, 0.5f},{ 0.5f,-0.5f, 0.5f},
        { 0.5f, 0.5f, 0.5f},{-0.5f, 0.5f, 0.5f}
    };

    mat4_t scale     = mat4_scale(2.0f, 2.0f, 2.0f);
    mat4_t rotate    = mat4_rotate_xyz(0.5f, 0.5f, 0.0f);
    mat4_t translate = mat4_translate(0.0f, 0.0f, -5.0f);

    float left = -1, right = 1, bottom = -1, top = 1, near = 1, far = 10;
    mat4_t proj = {0};
    proj.m[0][0] = (2 * near) / (right - left);
    proj.m[1][1] = (2 * near) / (top - bottom);
    proj.m[0][2] = (right + left) / (right - left);
    proj.m[1][2] = (top + bottom) / (top - bottom);
    proj.m[2][2] = -(far + near) / (far - near);
    proj.m[2][3] = -(2 * far * near) / (far - near);
    proj.m[3][2] = -1;

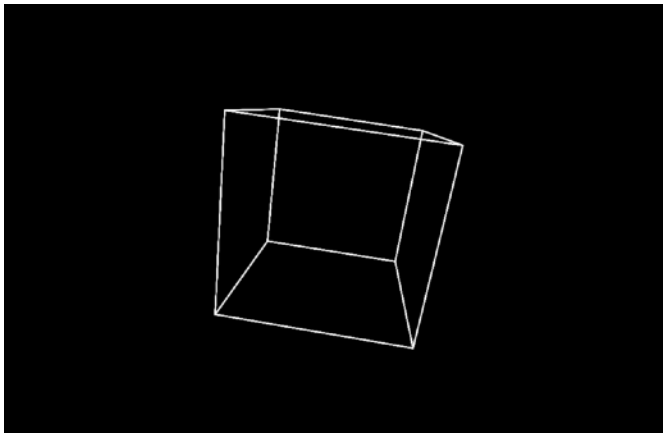
    for (int i = 0; i < 8; i++) {
        vec3_t p = cube[i];
        p = multiply_point(scale, p);
        p = multiply_point(rotate, p);
        p = multiply_point(translate, p);

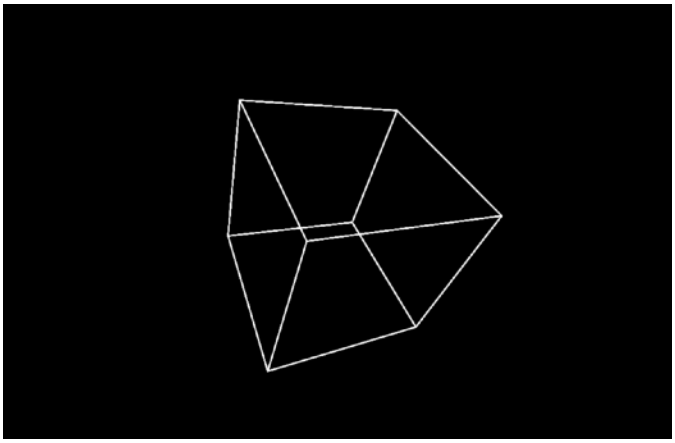
        vec3_t projected = multiply_point(proj, p);

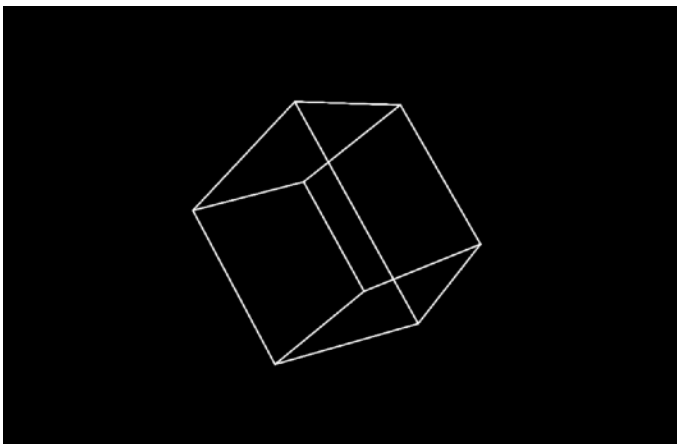
        printf(
          "Vertex %d: world=(%.2f,%.2f,%.2f)  proj=(%.2f,%.2f)\n",
          i, p.x, p.y, p.z, projected.x, projected.y
        );
    }

    return 0;
}
```

The multiply_point function applies a 4x4 matrix transformation to a 3D vector, handling the perspective division for proper projection. In the main function, an array defines the eight corners of a cube centered at the origin. The code then creates transformation matrices for scaling (doubling the cube's size), rotating (about the x and y axes), and translating (moving the cube 5 units away along the z-axis). Next, it sets up the parameters for a perspective projection matrix, defining the viewing frustum. For each cube vertex, the code applies scaling, rotation, translation, and finally the projection transformation, printing both the transformed 3D coordinates and the resulting 2D projected coordinates. The program doesn't actually draw the cube but demonstrates how each transformation affects the cube's vertices, showing the process of moving from 3D world space to 2D screen space.







TASK 3

```
void project_vertex(vec3_t vertex, mat4_t model, mat4_t view, mat4_t proj,
                    int width, int height, int *x, int *y, float *depth) {
    // Apply model, view, and projection matrices
    vec3_t world = mat4_mul_vec3(model, vertex);
    vec3_t camera = mat4_mul_vec3(view, world);
    vec3_t projected = mat4_mul_vec3(proj, camera);

    if (projected.z == 0) projected.z = 0.0001f;
    float f = 1.0f / -projected.z;

    float px = projected.x * f;
    float py = projected.y * f;

    *x = (int)((px + 1.0f) * 0.5f * width);
    *y = (int)((1.0f - py) * 0.5f * height);
    *depth = -projected.z;
}
```

The project_vertex function transforms a 3D point (vertex) from model space all the way to 2D screen coordinates using the standard 3D graphics pipeline: model, view, and projection transformations. It first applies the model matrix to move the vertex into world space, then the view matrix to move it into camera space, and finally the projection matrix to bring it into clip space. After projection, the function performs a perspective divide by the depth (z) to obtain normalized device coordinates (px, py). These are then converted into actual screen-space pixel coordinates (x, y) using the screen's width and height. The y-coordinate is flipped to match the screen's top-left origin convention. It also stores the depth value (distance from the camera) for potential depth buffering. If the projected z is zero (to avoid division by zero), a small value is substituted. This function is fundamental in rendering 3D objects onto a 2D screen.

```
int clip_to_circular_viewport(int width, int height, int x, int y) {
    int cx = width / 2, cy = height / 2;
    int r = (width < height ? width : height) / 2;
    int dx = x - cx, dy = y - cy;
    return (dx*dx + dy*dy) <= (r * r);
}
```

The clip_to_circular_viewport` function checks whether a given screen coordinate (x, y) lies inside a circular viewport centered on the screen. It first computes the center of the screen (cx, cy) and defines the radius r as half the smaller of the screen's width or height, ensuring the circle fits within the screen bounds. It then calculates the squared distance from the point (x, y) to the center using the Euclidean distance formula (without the square root for efficiency). If this distance is less than or equal to the square of the radius, the point lies inside the circular viewport and the function returns 1 (true); otherwise, it

returns 0 (false). This is useful for rendering within a circular view area, like in a radar or minimap.

```c
void render_wireframe(vec3_t *vertices, int v_count, int edges[][2], int e_count,
                      mat4_t model, mat4_t view, mat4_t proj,
                      int width, int height,
                      void (*draw_line)(int, int, int, int, unsigned char)) {
    int *xs = malloc(v_count * sizeof(int));
    int *ys = malloc(v_count * sizeof(int));
    float *zs = malloc(v_count * sizeof(float));

    for (int i = 0; i < v_count; i++) {
        project_vertex(vertices[i], model, view, proj, width, height, &xs[i], &ys[i], &zs[i]);
    }

    for (int i = 0; i < e_count; i++) {
        int a = edges[i][0], b = edges[i][1];
        int x0 = xs[a], y0 = ys[a], x1 = xs[b], y1 = ys[b];

        if (!clip_to_circular_viewport(width, height, x0, y0)) continue;
        if (!clip_to_circular_viewport(width, height, x1, y1)) continue;

        draw_line(x0, y0, x1, y1, 255);
    }

    free(xs); free(ys); free(zs);
}
```

The `render_wireframe` function renders a **3D wireframe model** onto a 2D screen by drawing lines between connected vertices. It takes an array of 3D vertices, a list of edges (pairs of vertex indices), transformation matrices (`model`, `view`, and `proj`), screen dimensions, and a `draw_line` function pointer to handle the actual drawing.

First, it allocates temporary arrays to store the projected screen-space `x`, `y` coordinates and depth (`z`) for each vertex. It then loops over all vertices, projecting them from 3D world space to 2D screen space using the `project_vertex` function. After projection, it iterates over all edges, retrieves the projected screen coordinates for each edge's endpoints, and uses `clip_to_circular_viewport` to check if both endpoints lie within a circular viewport (centered on the screen). If they do, it calls `draw_line` to render the line segment on the screen with a brightness value of 255. Finally, it frees the dynamically allocated memory for the temporary coordinate arrays. This function effectively implements a basic and efficient 3D wireframe renderer with viewport clipping.

3.3.2 DEMO

```
#define C0 0.809016994f
#define C1 1.618033988f
#define C2 1.809016994f
#define C3 2.118033988f
#define C4 2.427050983f
static const vec3_t verts[60] = {
 /* 0-11  */ { 0.5f,0, C4}, { 0.5f,0,-C4}, {-0.5f,0, C4}, {-0.5f,0,-C4},
              { C4, 0.5f,0}, { C4,-0.5f,0}, {-C4, 0.5f,0}, {-C4,-0.5f,0},
              {0, C4, 0.5f}, {0, C4,-0.5f}, {0,-C4, 0.5f}, {0,-C4,-0.5f},
 /* 12-23 */ { 1, C0, C3},{ 1, C0,-C3},{ 1,-C0, C3},{ 1,-C0,-C3},
              {-1, C0, C3},{-1, C0,-C3},{-1,-C0, C3},{-1,-C0,-C3},
              { C3, 1, C0},{ C3, 1,-C0},{ C3,-1, C0},{ C3,-1,-C0},
 /* 24-35 */ {-C3, 1, C0},{-C3, 1,-C0},{-C3,-1, C0},{-C3,-1,-C0},
              { C0, C3, 1},{ C0, C3,-1},{ C0,-C3, 1},{ C0,-C3,-1},
              {-C0, C3, 1},{-C0, C3,-1},{-C0,-C3, 1},{-C0,-C3,-1},
 /* 36-47 */ { 0.5f, C1, C2},{ 0.5f, C1,-C2},{ 0.5f,-C1, C2},{ 0.5f,-C1,-C2},
              {-0.5f, C1, C2},{-0.5f, C1,-C2},{-0.5f,-C1, C2},{-0.5f,-C1,-C2},
              { C2, 0.5f, C1},{ C2, 0.5f,-C1},{ C2,-0.5f, C1},{ C2,-0.5f,-C1},
 /* 48-59 */ {-C2, 0.5f, C1},{-C2, 0.5f,-C1},{-C2,-0.5f, C1},{-C2,-0.5f,-C1},
              { C1, C2,0.5f},{ C1, C2,-0.5f},{ C1,-C2,0.5f},{ C1,-C2,-0.5f},
              {-C1, C2,0.5f},{-C1, C2,-0.5f},{-C1,-C2,0.5f},{-C1,-C2,-0.5f}
};
```

This defines 60 fixed 3D points (vec3_t verts[60]) that form the vertices of a soccer ball-shaped wireframe, based on a truncated icosahedron. The constants `C0` to `C4` are derived from the golden ratio and related geometric values to ensure symmetry and spherical shape. The vertices are arranged in groups to define the core structure and detailed face connections of the ball. This array is used in rendering by connecting the points with edges, allowing the model to be rotated and displayed within a circular viewport.

```
static const int faces[32][6] = {
    {0,2,18,42,38,14},{1,3,17,41,37,13},{2,0,12,36,40,16},{3,1,15,39,43,19},
    {4,5,23,47,45,21},{5,4,20,44,46,22},{6,7,26,50,48,24},{7,6,25,49,51,27},
    {8,9,33,57,56,32},{9,8,28,52,53,29},{10,11,31,55,54,30},{11,10,34,58,59,35},
    {12,44,20,52,28,36},{13,37,29,53,21,45},{14,38,30,54,22,46},{15,47,23,55,31,39},
    {16,40,32,56,24,48},{17,49,25,57,33,41},{18,50,26,58,34,42},{19,43,35,59,27,51},
    {0,14,46,44,12,-1},{1,13,45,47,15,-1},{2,16,48,50,18,-1},{3,19,51,49,17,-1},
    {4,21,53,52,20,-1},{5,22,54,55,23,-1},{6,24,56,57,25,-1},{7,27,59,58,26,-1},
    {8,32,40,36,28,-1},{9,29,37,41,33,-1},{10,30,38,42,34,-1},{11,35,43,39,31,-1}
};
```

This array defines the 32 polygonal faces of the soccer ball wireframe using indices into the verts array. The first 20 entries represent hexagonal faces, each using 6 vertex indices. The last 12 entries represent pentagonal faces, padded with -1 as a placeholder since they have only 5 vertices. Each row in the faces array lists the vertices that form the boundary of a face, ordered for consistent edge connections. This structure allows the renderer to loop through the faces and draw the appropriate wireframe edges between the connected vertices.

```c
/* ---- edge list construction ---- */
int generate_soccer_ball(int out[90][2]){
    int cnt=0;
    for(int f=0;f<32;f++){
        int prev = faces[f][5]>=0 ? faces[f][5] : faces[f][4];
        for(int i=0;i<6;i++){
            int cur=faces[f][i]; if(cur<0) break;
            int a=prev<cur?prev:cur, b=prev<cur?cur:prev;
            int dup=0; for(int e=0;e<cnt;e++) if(out[e][0]==a && out[e][1]==b){dup=1;break;}
            if(!dup){ out[cnt][0]=a; out[cnt][1]=b; cnt++; }
            prev=cur;
        }
    }
    return cnt; /* should be 90 */
}
```

This function generate_soccer_ball constructs the unique list of edges for the soccer ball wireframe by analyzing the faces array. Each face (whether hexagon or pentagon) consists of a loop of vertices, and the function extracts each edge between consecutive vertices. It ensures no duplicate edges by checking if an edge (stored as a pair of sorted vertex indices) already exists in the out array before adding it. The prev and cur variables track connected vertex pairs, wrapping around the end of the face. The function returns the total number of unique edges, which should be 90 for a truncated icosahedron.

```c
#include "canvas.h"
void draw_edge(canvas_t *cv,int ax,int ay,int bx,int by){
    draw_line_f(cv,(float)ax,(float)ay,(float)bx,(float)by,1.0f);
}

int main(void){
    /* create canvas */
    canvas_t *cv = create_canvas(WIDTH,HEIGHT);

    /* build unique edge list */
    int edges[90][2]; int ecount = generate_soccer_ball(edges);

    /* basic camera / projection */
    mat4_t view = mat4_identity();
    mat4_t trans = mat4_translate(0,0,-5);

    float fov = 90*(M_PI/180.0f), asp=(float)WIDTH/HEIGHT;
    float near=0.1f, far=100; float f = 1.0f/tanf(fov*0.5f);
    mat4_t proj = mat4_identity();
    proj.m[0][0]=f/asp; proj.m[1][1]=f;
    proj.m[2][2]=far/(far-near); proj.m[2][3]=-far*near/(far-near);
    proj.m[3][2]=1; proj.m[3][3]=0;
```

```
    /* animation loop */
    for(int frame=0; frame<NFRAMES; ++frame){
        clear_canvas(cv);

        float ang = frame*0.03f;
        mat4_t model = mat4_mul(trans, mat4_rotate_xyz(ang, ang*0.6f, ang*0.3f));

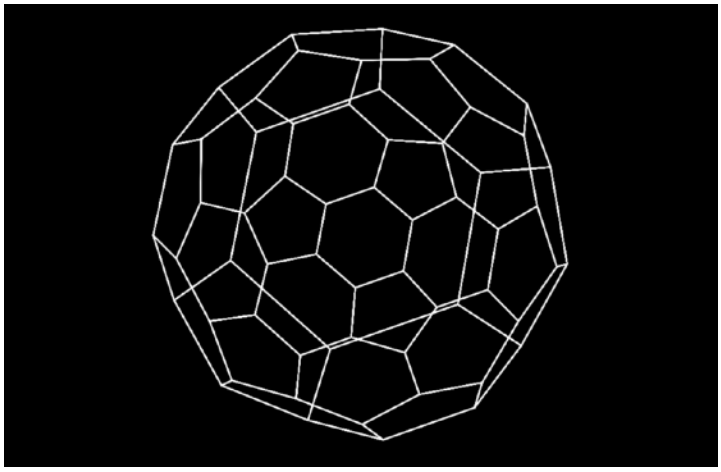
        /* project 60 vertices */
        int sx[60], sy[60]; float sz[60];
        for(int v=0; v<60; ++v)
            project_vertex(verts[v],model,view,proj,WIDTH,HEIGHT,&sx[v],&sy[v],&sz[v]);

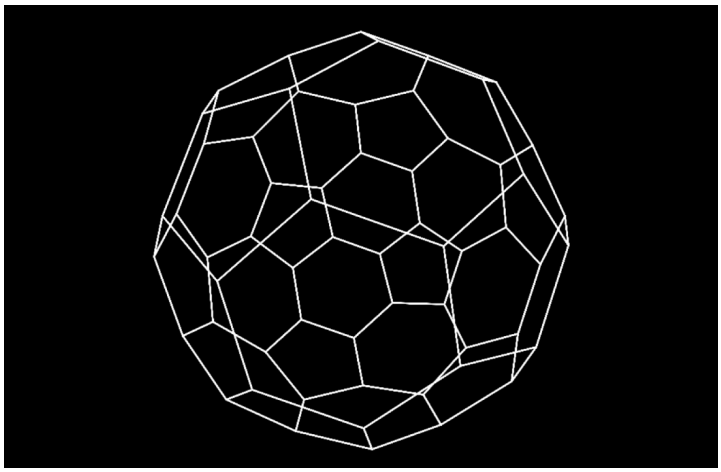
        /* draw 90 edges */
        for(int e=0; e<ecount; ++e){
            int a=edges[e][0], b=edges[e][1];
            draw_edge(cv, sx[a], sy[a], sx[b], sy[b]);
        }

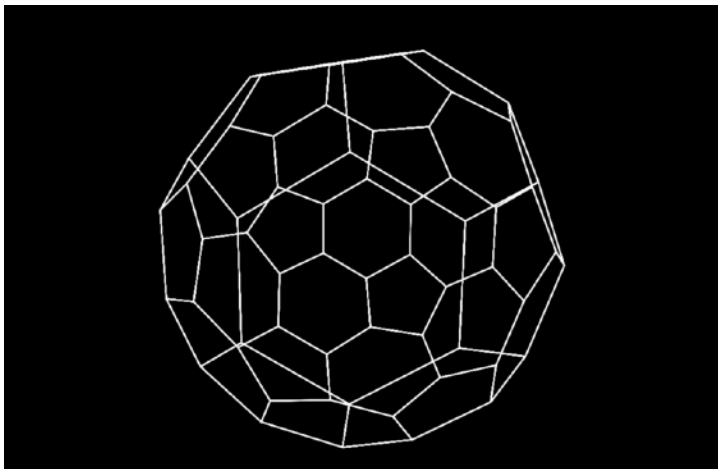
        save_canvas_pgm(cv,frame);
        printf("Saved soccer%04d.pgm\n",frame);
    }

    destroy_canvas(cv);
    return 0;
}
```

This is the main function of making soccer ball. First, it builds up a list of all the unique edges between the faces of the ball, making sure not to double up any edge, which is kinda tricky because each face shares edges with others. Then, it sets up a canvas to draw on and does some camera math so the ball looks 3D and sits a bit away from the camera, not right up in your face. It uses some projection math to turn the 3D points into 2D screen coordinates, so you can see the ball spinning. In the animation loop, for each frame, it rotates the ball a bit, projects all 60 vertices to screen space, and draws all 90 edges between them, making the wireframe. After drawing, it saves the frame as an image file, so you can see the animation if you look at the images in order. At the end, it cleans up the canvas. It's not super fancy, but it gets the job done for showing a spinning soccer ball made out of lines.







TASK 4

```
// Lambert lighting: average dot product of edge direction with multiple lights
float lambert_intensity(vec3_t edge_dir, vec3_t* lights, int light_count) {
    float total = 0;
    for (int i = 0; i < light_count; i++) {
        float dot = edge_dir.x * lights[i].x + edge_dir.y * lights[i].y + edge_dir.z * lights[i].z;
        total += fmaxf(0.0f, dot);
    }
    return total / light_count;
}
```

Lambert Lighting: Lambert lighting is done by calculating the brightness of an edge based on the angle between the edge direction and the light direction. This is done using the dot product of their normalized vectors, and then taking the maximum of zero and that dot product to ensure only edges facing the light are bright. For multiple lights, the intensities from each light are summed to get the final brightness. This simple calculation simulates diffuse lighting where brightness depends on how directly the surface faces the light.

```
vec3_t bezier3(vec3_t p0, vec3_t p1, vec3_t p2, vec3_t p3, float t) {
    float u = 1 - t;
    float b0 = u*u*u, b1 = 3*u*u*t, b2 = 3*u*t*t, b3 = t*t*t;
    return (vec3_t){
        p0.x*b0 + p1.x*b1 + p2.x*b2 + p3.x*b3,
        p0.y*b0 + p1.y*b1 + p2.y*b2 + p3.y*b3,
        p0.z*b0 + p1.z*b1 + p2.z*b2 + p3.z*b3
    };
}
```

Bézier animations are created by moving an object smoothly along a Bézier curve defined by control points. The process works by calculating the object's position for each frame based on a parameter that goes from 0 to 1, representing the progress along the curve. For a cubic Bézier curve with four control points, the position at each t is found by recursively interpolating between these points: first between the control points themselves, then between the resulting points, until a single point on the curve is obtained. This point determines the object's position at that moment. By incrementing t steadily over time, the object moves smoothly along the curve from start to end. This method allows precise control over the motion path and speed, creating natural and fluid animations

3.4.2 DEMO

```c
typedef struct {
    int a;
    int b;
} edge_t;

typedef struct {
    vec3_t position;
    vec3_t normal;
} vertex_t;

// Cube vertices with normals (24 vertices, 4 per face)
static vertex_t cube_vertices[24] = {
    {{-1, -1, -1}, {0, 0, -1}}, {{1, -1, -1},  {0, 0, -1}}, {{1, 1, -1},   {0, 0, -1}}, {{-1, 1, -1},  {0, 0, -1}},
    {{-1, -1, 1},  {0, 0, 1}},  {{1, -1, 1},   {0, 0, 1}},  {{1, 1, 1},    {0, 0, 1}},  {{-1, 1, 1},   {0, 0, 1}},
    {{-1, -1, -1}, {-1, 0, 0}}, {{-1, 1, -1},  {-1, 0, 0}}, {{-1, 1, 1},   {-1, 0, 0}}, {{-1, -1, 1},  {-1, 0, 0}},
    {{1, -1, -1},  {1, 0, 0}},  {{1, 1, -1},   {1, 0, 0}},  {{1, 1, 1},    {1, 0, 0}},  {{1, -1, 1},   {1, 0, 0}},
    {{-1, -1, -1}, {0, -1, 0}}, {{1, -1, -1},  {0, -1, 0}}, {{1, -1, 1},   {0, -1, 0}}, {{-1, -1, 1},  {0, -1, 0}},
    {{-1, 1, -1},  {0, 1, 0}},  {{1, 1, -1},   {0, 1, 0}},  {{1, 1, 1},    {0, 1, 0}},  {{-1, 1, 1},   {0, 1, 0}},
};

static edge_t cube_edges[] = {
    {0,1},{1,2},{2,3},{3,0},
    {4,5},{5,6},{6,7},{7,4},
    {8,9},{9,10},{10,11},{11,8},
    {12,13},{13,14},{14,15},{15,12},
    {16,17},{17,18},{18,19},{19,16},
    {20,21},{21,22},{22,23},{23,20}
};
```

Here, the edge_t struct is used to define an edge by storing two vertex indices that tells which vertices are connected. The vertex_t struct holds both the position of a vertex in 3D space and its normal vector, which is important for lighting calculations. The cube is represented by 24 vertices in the cube_vertices array, because each face of the cube has 4 vertices and there are 6 faces total. Each vertex also has a normal pointing outward from the face it belongs to, helping to create correct shading. The cube_edges array then lists pairs of these vertex indices to describe all the edges of the cube, basically defining its wireframe structure. Together, these structs and arrays make it possible to represent the cube's shape and how it interacts with light for rendering and animation.

```c
static const int cube_edge_count = sizeof(cube_edges) / sizeof(cube_edges[0]);

vec3_t cubic_bezier(vec3_t p0, vec3_t p1, vec3_t p2, vec3_t p3, float t) {
    float u = 1 - t;
    float tt = t*t;
    float uu = u*u;
    float uuu = uu * u;
    float ttt = tt * t;

    vec3_t result;
    result.x = uuu * p0.x + 3 * uu * t * p1.x + 3 * u * tt * p2.x + ttt * p3.x;
    result.y = uuu * p0.y + 3 * uu * t * p1.y + 3 * u * tt * p2.y + ttt * p3.y;
    result.z = uuu * p0.z + 3 * uu * t * p1.z + 3 * u * tt * p2.z + ttt * p3.z;
    return result;
}

void draw_filled_circle(canvas_t *c, int cx, int cy, int radius, float intensity) {
    for (int y = -radius; y <= radius; y++) {
        for (int x = -radius; x <= radius; x++) {
            if (x*x + y*y <= radius*radius) {
                int px = cx + x;
                int py = cy + y;
                if (px >= 0 && px < c->width && py >= 0 && py < c->height) {
                    float old = c->pixels[py][px];
                    c->pixels[py][px] = old + intensity > 1.0f ? 1.0f : old + intensity;
                }
            }
        }
    }
}
```

By defining cube_edge_count as the total number of edges in the cube by dividing the size of the whole cube_edges array by the size of one edge element, so it automatically adapts if edges change. The cubic_bezier function calculates a point on a cubic Bézier curve at a given time `t` between 0 and 1, using four control points p0 to p3. It computes blending factors based on powers of `t` and 1 - t, then combines the control points weighted by these factors to get a smooth interpolated position in 3D space. The `draw_filled_circle` function draws a filled circle on a canvas by checking every pixel within a square bounding box around the circle's center and radius, and if the pixel lies inside the circle, it increases its intensity but clamps it to a maximum of 1.0 to avoid overflow. This circle is useful for drawing things like the light source on the screen. Overall, these functions help in smoothly animating objects along curved paths and rendering visual elements with proper brightness.

```
void render_cube_edges(canvas_t* canvas, mat4_t model, mat4_t view, mat4_t proj, vec3_t light) {
    float edge_intensity[cube_edge_count];
    for (int e = 0; e < cube_edge_count; e++) {
        vertex_t v0 = cube_vertices[cube_edges[e].a];
        vertex_t v1 = cube_vertices[cube_edges[e].b];

        mat4_t rot_only = model;
        rot_only.m[0][3] = rot_only.m[1][3] = rot_only.m[2][3] = 0.0f;

        vec3_t n0 = mat4_mul_vec3(rot_only, v0.normal);
        vec3_t n1 = mat4_mul_vec3(rot_only, v1.normal);

        n0 = vec3_normalize_fast(n0);
        n1 = vec3_normalize_fast(n1);

        vec3_t p0 = mat4_mul_vec3(model, v0.position);
        vec3_t p1 = mat4_mul_vec3(model, v1.position);
        vec3_t midpoint = {(p0.x + p1.x) * 0.5f, (p0.y + p1.y) * 0.5f, (p0.z + p1.z) * 0.5f};

        vec3_t light_dir = vec3_normalize_fast((vec3_t){light.x - midpoint.x, light.y - midpoint.y, light.z - midpoint.z});

        vec3_t lights[1] = {light_dir};
        float i0 = lambert_intensity(n0, lights, 1);
        float i1 = lambert_intensity(n1, lights, 1);
        edge_intensity[e] = (i0 + i1) * 0.5f;
    }

    int xs[24], ys[24];
    float zs[24];
    for (int i = 0; i < 24; i++) {
        project_vertex(cube_vertices[i].position, model, view, proj, WIDTH, HEIGHT, &xs[i], &ys[i], &zs[i]);
    }

    for (int e = 0; e < cube_edge_count; e++) {
        float brightness = edge_intensity[e];
        float thickness = 0.5f + brightness * 3.0f;
        draw_line_f(canvas, xs[cube_edges[e].a], ys[cube_edges[e].a], xs[cube_edges[e].b], ys[cube_edges[e].b], thickness);
    }
}
```

The render_cube_edges function is used for drawing the cube's edges on the canvas with proper lighting and thickness based on their brightness. It first calculates the lighting intensity for each edge by taking the normals of the two vertices that form the edge, transforming and normalizing them using the rotation part of the model matrix. Then it finds the midpoint of the edge in world space and computes the direction from this midpoint to the light source. Using Lambert lighting, it calculates the light intensity at both vertices and averages them to get the edge's brightness. Next, it projects all 24 cube vertices from 3D space to 2D screen coordinates using the model, view, and projection matrices. Finally, it draws each edge as a line on the canvas, adjusting the line thickness

according to the brightness so brighter edges appear thicker. This approach combines geometric transformations, lighting calculations, and projection to render a visually realistic wireframe cube with dynamic shading.

```
int main() {
    const vec3_t light = {0, 0, 0}; // Light source at origin
    vec3_t P0 = {-5, 0, -10}, P1 = {-5, 5, -10}, P2 = {5, 5, -10}, P3 = {-5, 0, -10};
    vec3_t P0 = {-5, 0, -10} ,P1 = {-5, 5, -10} ,P2 = {5, 5, -10} ,P3 = P0;

    mat4_t view = mat4_identity();

    float fov = M_PI / 2;
    float aspect = (float)WIDTH / HEIGHT;
    float near = 0.1f, far = 100.0f;
    float f = 1.0f / tanf(fov / 2);
    mat4_t proj = mat4_identity();
    proj.m[0][0] = f / aspect;
    proj.m[1][1] = f;
    proj.m[2][2] = far / (far - near);
    proj.m[2][3] = -far * near / (far - near);
    proj.m[3][2] = 1;
    proj.m[3][3] = 0;

    for (int frame = 0; frame < FRAMES; frame++) {
        float t1 = (float)frame / (FRAMES - 1);
        float t2 = t1 - 0.5f; // second cube offset by half the path
        if (t2 < 0.0f) t2 += 1.0f; // wrap around

        vec3_t pos1 = cubic_bezier(P0, P1, P2, P3, t1);
        vec3_t pos2 = cubic_bezier(P0, P1, P2, P3, t2);

        canvas_t* canvas = create_canvas(WIDTH, HEIGHT);
        clear_canvas(canvas);

        // First cube model: translate and rotate
        mat4_t model1 = mat4_mul(
            mat4_translate(pos1.x, pos1.y, pos1.z),
            mat4_rotate_xyz(t1 * 2 * M_PI, t1 * 2 * M_PI, 0)
        );

        // Second cube model: translate and rotate
        mat4_t model2 = mat4_mul(
            mat4_translate(pos2.x, pos2.y, pos2.z),
            mat4_rotate_xyz(t2 * 2 * M_PI, t2 * 2 * M_PI, 0)
        );

        render_cube_edges(canvas, model1, view, proj, light);
        render_cube_edges(canvas, model2, view, proj, light);

        // Draw light source as white ball at origin
        int light_x, light_y;
        float light_depth;
        project_vertex(light, mat4_identity(), view, proj, WIDTH, HEIGHT, &light_x, &light_y, &light_depth);
        draw_filled_circle(canvas, light_x, light_y, 6, 1.0f);

        save_canvas_pgm(canvas, frame);
        destroy_canvas(canvas);

        printf("Frame %3d saved\n", frame);
    }

    return 0;
}
```

The main function is where everything comes together to create the animation. First, it sets the light source at the center and defines four points that make up a smooth looping

path using a Bézier curve, the start and end points are the same so the animation loops nicely without any jumps. It sets up the camera view and projection to make the 3D scene look right on the screen. Then, for each frame in the animation, it calculates where two cubes should be along the path at that moment in time, making sure the second cube is halfway behind the first to keep their movement synced but staggered. Each cube's position is combined with a rotation so they spin as they move. The cubes are drawn on a canvas with proper lighting and shading, and the light source itself is shown as a bright circle. After drawing, the frame is saved as an image, and the canvas is cleared for the next frame. This loop repeats for all frames, producing a smooth, continuous animation of two cubes moving and rotating together along the same path.

